PICK: An SRAM-based Processing-in-Memory Accelerator for K-Nearest-Neighbor Search in Point Clouds

Chen Nie[1,2], Chao Jiang[3], Liming Xiao[3], Weifeng Zhang[3], Zhezhi He[1,2]
[1]School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China
[2]Intelligent Computing Research Group (ICRG), Shanghai Jiao Tong University, China [3]Lenovo Research, Beijing, China
{chen.nie, zhezhi.he}@sjtu.edu.cn {jiangchao13, xiaolm, weifengz}@lenovo.com

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132408

***Abstract*—K-nearest neighbor (kNN) search is a fundamental operation in various point cloud applications, such as autonomous driving. However, the heavy computational intensity and memory demands of kNN search pose significant challenges for efficient implementation, especially in resource-constrained scenarios. To address these challenges, we propose PICK, a processing-in-memory (PIM) architecture designed to accelerate kNN search in point cloud applications. PICK leverages bit-serial-based PIM (BS-PIM) and customized circuits to efficiently handle key operations of kNN search: distance calculation and top-k selection. The run-time off-chip access is eliminated thanks to the large on-chip memory. For distance calculation, we introduce a bit-width clipping technique to reduce the latency of bit-serial execution with negligible accuracy degradation, providing flexible trade-offs between performance and precision. Besides, we propose a filtering-and-selection strategy that realizes approximately constant time complexity for arbitrary values of k. Furthermore, a two-stage pipeline is implemented to parallelize distance calculation and top-k search, effectively hiding latency and improving throughput. According to our experiments, PICK achieves $4.17\times$ speedup and a $4.42\times$ energy saving over the state-of-the-art design.**

***Index Terms*—Processing-in-Memory, SRAM, KNN Search**

I. Introduction

Recent advancements in artificial intelligence have significantly driven the adoption of three-dimensional (3D) visual data, commonly known as point clouds. Generated by specialized sensors such as LiDAR [1], point clouds consist of sets of points representing the surfaces and properties of surrounding objects and environments. Their versatility has led to extensive use across various domains, including autonomous driving [2], robotics [3], and virtual reality [4]. These applications demand high accuracy while adhering to stringent performance constraints, such as low latency for real-time processing and minimal power consumption for edge deployments.

To meet these demands, significant research has focused on optimizing the *k-nearest neighbor* (kNN) search, a core operation in point cloud processing. KNN search identifies k closest reference points to a query point and underpins critical algorithms such as simultaneous localization and mapping (SLAM) [5], point registration [6], object recognition [7], and scene segmentation [8]. KNN search comprises two computationally intensive operators: *distance calculation* and *top-k search*, requiring extensive arithmetic computations and comparisons. Prior works [9], [10] have revealed that *kNN search can account for up to 80% of the latency in real-world point cloud applications*, primarily due to the computationally intensive nature of distance calculation and top-k search. Therefore, optimizing kNN search is crucial to improving the performance of point cloud applications, particularly in real-time and resource-constrained environments.

Recently, several kNN-specific accelerators [10]–[17] have been developed to enhance computational efficiency and reduce latency in point cloud applications. Partition-based techniques, *e.g.* kd-trees in QuickNN [11], octrees in ParallelNN [12] and SimDiff [14], hashing [15], and voxelization [16], have been explored to minimize computation and memory access. However, these methods often compromise accuracy (*e.g.*, 80% in [12]) and introduce preprocessing overhead. JUNO [13] exploits sparsity and locality to accelerate approximate neighbor search but cannot ensure full accuracy. BitNN [10] uses bit-serial computation and early termination to enhance efficiency but suffers from excessive on-chip and off-chip memory access, which accounts for over 35% of total energy consumption. Despite these advancements, challenges remain in *further reducing computation latency and memory access while preserving satisfactory accuracy*.

In this work, we propose PICK, a novel processing-in-SRAM architecture tailored to efficiently accelerate kNN search for point cloud applications. PICK leverages bit-serial-based PIM (BS-PIM) and custom computing modules to optimize the fundamental operations of kNN search, namely, distance calculation and top-k search. The *in-situ* bit-serial computing approach of BS-PIM minimizes on-chip data movement and simplifies circuit design, enabling sufficient on-chip memory to eliminate runtime off-chip memory access entirely.

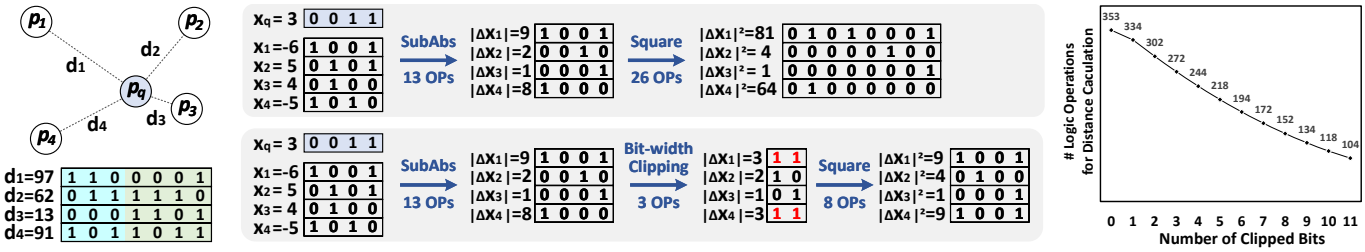
Furthermore, to address the high latency of bit-serial algorithms [18], we introduce a bit-width clipping technique that reduces latency with negligible accuracy degradation, allowing flexible performance-accuracy trade-offs for diverse use cases. For top-k search, we design an efficient filtering-and-selection strategy that achieves approximately constant time complexity for arbitrary k. Additionally, a two-stage pipeline parallelizes distance calculation and top-k search, hiding latency and improving throughput. Extensive evaluations on real-world point cloud datasets demonstrate that PICK achieves a $4.17\times$ speedup and $4.42\times$ energy savings compared to the state-of-the-art accelerator BitNN [10]. The key contributions of this work are summarized as:

- We propose PICK, the first PIM-based architecture to accelerate the kNN search in point clouds. The highly parallelized and in-situ computing approach greatly accelerates the computation and reduces both on-chip and off-chip data movements.
- We propose a bit-width clipping method to reduce the latency and energy of distance calculation, with negligible accuracy loss.
- We propose a filtering-and-selection-based search method to efficiently perform top-k search for arbitrary values of k, enabling optimal performance across a wide range of application scenarios.
- By comparison, PICK achieves $4.17\times$ speedup and $4.42\times$ energy saving over the state-of-the-art accelerator BitNN.

II. Background

Point Cloud The point cloud is an unordered collection of discrete points, represented as $\mathbb{P} = \{\boldsymbol{p}_i\}$, where $\boldsymbol{p}_i = (x_i, y_i, z_i)$ specifies the Cartesian coordinates of the i-th point in the set $\mathbb{P}$. Typically, point clouds are continuously captured by specialized devices (*e.g.*, LiDAR and 3D scanner), enabling the perception and reconstruction of surrounding environments. Modern LiDAR sensors [19] can generate

This work is partially supported by the National Natural Science Foundation of China under Grant 62102257 and National Key R&D Program of China under Grant 2022YFB4500200. Corresponding Author: Zhezhi He.



(a) Example of 1-NN (b) Distance calculation without (top) and with (bottom) bit-width clipping (c) Reduction of operations

Fig. 1: **Motivation for bit-width clipping**. (a) Example of 1-NN (kNN with $k = 1$) search between a query point $\boldsymbol{p}_q$ and four reference points $\{\boldsymbol{p}_1, \boldsymbol{p}_2, \boldsymbol{p}_3, \boldsymbol{p}_4\}$, where $\boldsymbol{p}_3$ is identified nearest with a distance d_3 which can be accurately represented using only 4 bits (in green). (b) Comparison of distance calculation along the x-dimension, shown without (top) and with (bottom) bit-width clipping. Bit-width clipping reduces the number of bit-serial logic operations (OPs) and memory usage, while maintaining $\boldsymbol{p}_3$ as the nearest point. (c) As more bits are clipped (initially 16-bit), the distance calculation involves fewer logic operations, leading to a reduction in computational latency.

up to 460k points per frame at a frame rate of 20Hz (*i.e.*, 50ms per frame), delivering rich spatial information that underpins a wide range of applications. Recent point cloud datasets [20]–[23] cover diverse scenarios, including objects, indoor and outdoor environments, and geographic landscapes, further broadening the utility and applicability of point cloud data.

K-nearest-neighbor (kNN) search. The kNN search is an extensively employed algorithm to identify the top-k closest reference points to a given query point, in point cloud applications. This process consists of two primary operations: *distance calculation* and *top-k search*. The distance calculation computes the squared Euclidean distance between a reference point $\boldsymbol{p}_i = (x_i, y_i, z_i)$ and a given query point $\boldsymbol{p}_\mathrm{q} = (x_\mathrm{q}, y_\mathrm{q}, z_\mathrm{q})$, which can be expressed as $d = |x_i - x_\mathrm{q}|^2 + |y_i - y_\mathrm{q}|^2 + |z_i - z_\mathrm{q}|^2$. Once the distance d_i is calculated for each reference point $\boldsymbol{p}_i$ relative to the query point $\boldsymbol{p}_\mathrm{q}$, the top-$k$ search is performed to identify the k points with the shortest distances and retrieve corresponding indices as $\{i_1, i_2, \ldots, i_k\}$.

Bit-Serial-based Processing-in-Memory. Processing-in-memory (PIM) architectures [24]–[27] have been extensively investigated to accelerate various applications, offering reduced data movements and substantial computing parallelism. Bit-serial-based PIM (BS-PIM) is an extension of PIM that incorporates bit-serial algorithms [18], aiming to support general-purpose tasks. In bit-serial algorithms, multi-bit arithmetic functions (*e.g.*, addition, multiplication) are decomposed into multiple bit-wise logic operations (*e.g.*, `AND`, `OR`, `XOR`), which are executed sequentially over multiple cycles. To realize these operations efficiently, existing BS-PIM designs [28]–[32] employ bit-line computing technique [28] that enable parallel execution of bit-wise logic operations, and further extend support to arbitrary multi-bit arithmetic functions. In essence, BS-PIM facilitates *in-situ* scalar-vector [31] and vector-vector [29] arithmetic operations, making it a versatile architecture for diverse computational tasks.

III. Motivations

A. Minimizing Data Movements by Efficient PIM architecture

Problem. Memory access poses a significant bottleneck in optimizing kNN search, prominently impacting both latency and energy consumption. For instance, prior work [10] highlights that *non-continuous DRAM access accounts for over 80% of runtime* in QuickNN [11], while the on/off-chip data movements contribute to 35% of the energy consumption in BitNN [10]. To address these challenges, we propose to leverage PIM architecture to accelerate kNN search and minimize both on-chip and off-chip data movement.

Idea. Our approach incorporates customized computing circuits alongside the BS-PIM micro-architecture design from [28]–[31] to enable efficient in-memory kNN search. The large on-chip memory capacity in our design eliminates runtime off-chip memory access, while delicately optimized execution flows significantly reduce on-chip data movement and maximize computing parallelism. Furthermore, the bit-serial execution model inherent to BS-PIM facilitates energy-efficient *in-situ* arithmetic operations, thereby substantially lowering the energy consumption associated with kNN search.

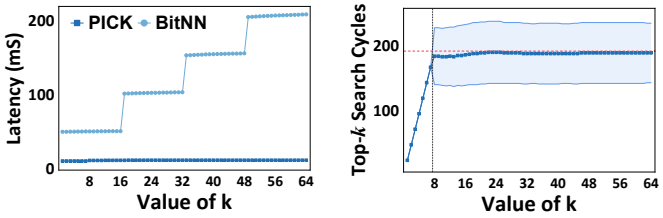
B. Optimizing Distance Calculation through Bit-width Clipping

Problem. Bit-serial algorithms support a wide range of arithmetic functions, however, at the cost of high latency. For instance, an N-bit unsigned integer multiplication requires $(N^2 + 5N - 2)$ logic operations [30], *i.e.*, consuming 334 clock cycles for 16-bit data. Despite substantial throughput achieved through extensive computing parallelism, the execution latency of BS-PIM remains considerably higher than specialized digital circuits, particularly for distance calculation in kNN search. Reducing the bit-width of point cloud coordinates (x, y, z) can only marginally improve the latency since a minimum of 14-bit is required to maintain accuracy, as in BitNN [10].

Idea. We propose *bit-width clipping* to clip the bit-width precision of multiplications in distance calculation, thereby reducing operational cost and improving latency, with minimal impact on accuracy. This method leverages the observation that the *smallest distance, indicating the nearest point, can be precisely represented with fewer bits*. For example, in the 1-NN scenario shown in Fig. 1(a), the nearest reference point $\boldsymbol{p}_3$ to the query point $\boldsymbol{p}_q$ has a distance d_3=13, which can be accurately represented using only the lower four bits. Using additional bits to accurately represent distances $\{d_1, d_2, d_4\}$ is unnecessary for identifying the nearest point but incurs more latency and energy.

Therefore, we clip the bit-width of squaring (*i.e.*, multiplication) in distance calculation to reduce operations and memory usage. The top of Fig. 1(b) shows distance calculation along the x-dimension without bit-width clipping, where $|\Delta x_i|^2$ is obtained through subtraction (*i.e.*, addition in 2's complement) and absolute value computation, followed by square operation. In contrast, the bottom portion of Fig. 1(b) applies bit-width clipping, where the bit-width of $|\Delta x_i|$ is clipped from 4-bit to 2-bit, greatly simplifying the subsequent square operation. Note that, the values of $|\Delta x_1|$ and $|\Delta x_4|$ exceed the representable range of 2-bit and are re-mapped to 3 (*i.e.*, the maximum value in 2-bit), to ensure they remain farther from the query point. Bit-width clipping preserves the accuracy of the kNN result, as $|\Delta x_i|^2$ remains the smallest value, while reducing 28% bit-wise logic operations.

In practical usage, where bit-widths are typically higher, bit-width clipping yields even greater improvements. We select an initial bit-width of 16 based on our analysis of four real-world datasets in Table I.



(a) Latency on KITTI at different k (b) Top-k search cycles at different k

Fig. 2: **Performance analysis at different k.** (a) The latency of PICK and BitNN [10] on KITTI dataset [20] for various k values (clipping width is 6-bit). (b) The cycles required by PICK for top-k search at different k. The vertical dashed line ($k = 7.5$) marks the boundary of small and large k, the shadow indicates variation.

Fig. 1(c) presents the number of operations at each clipping width, where we can achieve 56% reduction in operations by clipping 8-bit, with negligible accuracy degradation as proved in Section V.

C. Efficient Filtering-and-Selection Search for Arbitrary K

Problem. In kNN, k varies significantly w.r.t. different applications. For instance, point registration [6] often uses 1-NN, while classification [33] and segmentation [34] tasks require $k \in [3, 64]$. This variability in k poses a challenge for existing accelerators to achieve optimal performance. For example, BitNN [10] includes 16 top-k units, which support searching 16 nearest points in parallel for each iteration. When $k = 64$, four iterations are required with 4× latency as plotted in Fig. 2a, while also claimed to negatively affect accuracy [10]. Increasing the number of top-k units can mitigate latency growth, but it incurs hardware overhead and leads to underutilization for small k. In summary, *efficient support of arbitrary k is yet absent*. As the countermeasure, we introduce *filtering-and-selection-based top-k search to efficiently support arbitrary k*.

Idea. We first propose a selection-based method to implement top-k search of distances through k iterations of min-search. In each iteration, the nearest point with the smallest distance is identified, continuing this process for k times to obtain the k closest points. Notably, previously identified points are excluded from subsequent searches. This approach is efficient when k is small but suffers from linearly increasing latency as k grows, as in Fig. 2b.

Therefore, for larger values of k, we propose a filtering-based method to improve efficiency. Specifically, we introduce a threshold t to filter the distances d_i, and count the number of points within this threshold ($d_i < t$), denoted as n. By selecting an appropriate threshold t that satisfies $n = k$, the n points within the threshold correspond exactly to the top-k nearest points. However, in practice, achieving an exact match is often challenging. Thus, we employ a fast bisection algorithm to dynamically adjust the threshold t, iteratively refining it until n is close to k, such that $|n-k| < m$, where m is a variable tolerance. Once this condition is met, the selection-based method finalizes the process by including or excluding points as necessary to ensure an exact match with k. The optimal value of m and the boundary that distinguishes small and large k are determined through explorations. In KITTI dataset [20], we select $k = 7.5$ as the boundary and $m = 4$. Consequently, arbitrary top-k searches are accurately achieved with nearly constant time complexity, as shown in Fig. 2.

IV. Architecture of PICK

A. Architecture Overview

The architecture of PICK is illustrated in Fig. 3, comprising multiple components designed to efficiently perform kNN search. The

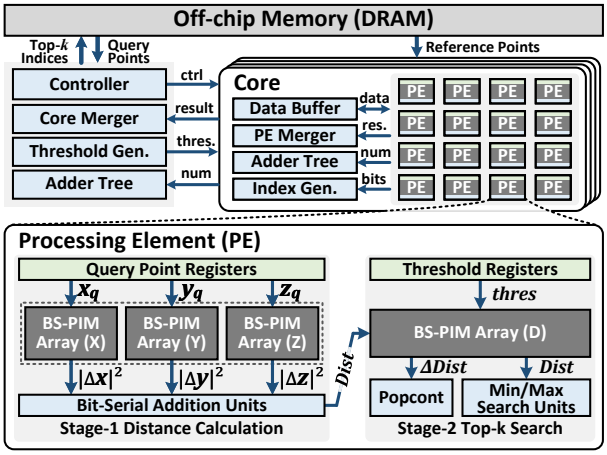


Fig. 3: **Overview of PICK architecture and dataflow.** The reference points are streamed from off-chip to the processing elements (PEs) of multiple cores for parallel computing. BS-PIM arrays cooperate with customized circuits to perform distance calculation and top-k search with a two-stage pipeline. The local search results are merged and the indices of the top-k nearest points are restored to the off-chip.

architecture features *multiple cores* dedicated to distance calculation and top-k search, a *core merger* for consolidating local search results from individual cores, a *threshold generator* and an *adder tree* for point filtering and counting (detailed in Section III-C), and a *controller* responsible for execution scheduling. Notably, the cores also function as on-chip storage for point cloud coordinates.

Core and PE. Each core integrates several key elements, including multiple processing elements (PEs), a data buffer with transposable access capabilities [28], a PE merger to aggregate search results from PEs, a local adder tree for point counting, and an index generator for retrieving the indices of searched points. At the core's foundation, the PE serves as the fundamental unit for kNN search. Each PE consists of four BS-PIM arrays dedicated to processing the x, y, and z dimensions, as well as distance computations. Additional components include registers for query points and thresholds, bit-serial-based units for arithmetic operations such as addition and min/max search, and a pop-count module for efficiently counting filtered points. This modular design ensures optimized performance and scalability for kNN search.

Dataflow of PICK, as annotated in Fig. 3, begins with point cloud data (coordinates) generated by sensors (*e.g.*, LiDAR) and stored in off-chip memory. Execution starts by streaming the reference points into the data buffers of multiple cores, where they are loaded into processing elements (PEs) in a transposed bit layout [28]. The reference points are stored in the BS-PIM arrays, while the coordinates of the query point are loaded into the PE registers.

Distance calculations for the x, y, and z dimensions are performed in parallel to compute $|\Delta x|^2$, $|\Delta y|^2$, and $|\Delta z|^2$, which are summed using bit-serial addition units and stored in the BS-PIM array dedicated to distance (Array D) for subsequent top-k search. For small values of k, the top-k nearest points are identified through k iterations of min-search, leveraging bit-serial min/max-search units, as detailed in Section III-C. For larger values of k, filtering is applied before selection, where distances are compared against a dynamical threshold, and the pop-count module counts the points within the threshold.

To optimize performance, a two-stage pipeline is employed to parallelize distance calculations and top-k search, to be described in Section IV-C. Since the number of reference points typically exceeds the capacity of a single PE, the data is distributed across multiple

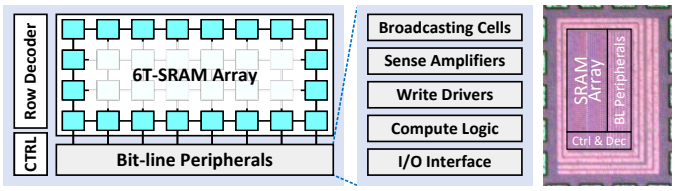


Fig. 4: **BS-PIM micro-architecture and micrograph of our demonstrated chip.** Parallel scalar/vector-vector bit-wise logic operations (AND, OR, XOR, and FA) are implemented utilizing bit-line computing technique [28] and integrated bit-line peripherals.

PEs and cores to enable parallel processing. Local search results, including values and indices, are hierarchically merged to produce the final result. Ultimately, the indices of the top-k nearest points are written back to off-chip memory.

B. Micro-architecture

1) BS-PIM Array: The micro-architecture of a single BS-PIM array, illustrated in Fig. 4, is designed to handle arithmetic operations (*e.g.*, addition, subtraction, multiplication) essential for kNN search. The circuit design of BS-PIM leverages optimizations from prior works [28]–[31], ensuring efficient and reliable performance.

At the core of the BS-PIM design is the 6T-SRAM array, which stores reference points or distances and enables parallel bit-wise logic operations (AND and NOR) through bit-line computing [28] and its associated bit-line peripherals. These peripherals are integral to both memory functionality and bit-serial computation. The broadcasting cells [31] facilitate the bit broadcasting of query points and distance thresholds, enabling concurrent scalar-vector arithmetic across all reference points and distances stored in the SRAM array.

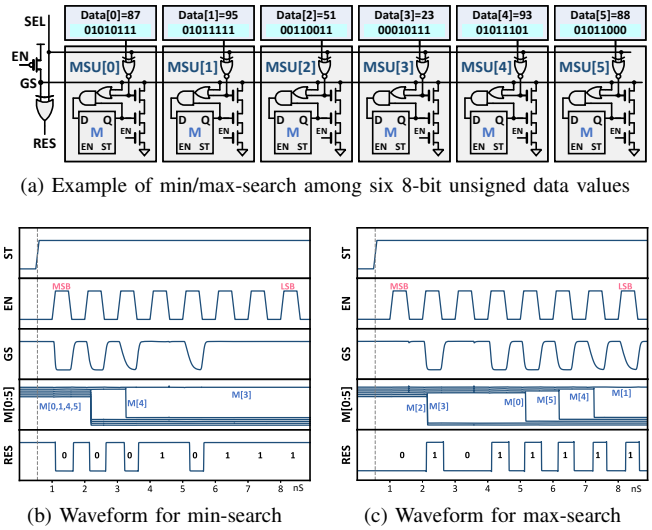
Additional components include sense amplifiers and write drivers, which support memory read and write operations. Compute logic modules handle post-processing tasks such as bit-wise addition and multiplexing, while the I/O interface manages data input and output. These elements collectively ensure seamless integration of memory and computation within the BS-PIM array.

The demonstrated BS-PIM array, as depicted in Fig. 4, can execute the described computations at a maximum frequency of 1.35 GHz, making it a highly efficient and scalable solution for kNN search.

2) Min/Max-Search Units: The micro-architecture of the min/max-search units is depicted in Fig. 5a, which implements bit-serial-based vector-wise min/max search by iteratively comparing bits from the most significant bit (MSB) to the least significant bit (LSB) and conditionally updating the latch states (M). An example with six 8-bit unsigned integer values is shown in Fig. 5 to illustrate the procedure.

In each iteration, one bit from all data items is read as a bit vector, which is subsequently processed by the search units. This work specifically addresses the min/max-search of unsigned integers, as squared Euclidean distances are inherently non-negative. In this context, minimum and maximum values are distinguished by the concentration of '0' and '1' bits near the MSB, respectively. Therefore, *min-search can be treated as a max-search with input bits inverted*.

To accommodate this, we design max-search circuits and employ XNOR gates to invert input bits based on the SEL signal, as shown in Fig. 5a. When SEL is set to low, the circuit performs min-search, identifying Data[3] as the minimum value (Fig. 5b). Conversely, setting SEL to high enables max-search, identifying Data[1] as the maximum value (Fig. 5c). In PEs, the min/max-search process follows the same procedure but is executed with higher parallelism. Additionally, the index generator, as described in Fig. 3, converts



(a) Example of min/max-search among six 8-bit unsigned data values

(b) Waveform for min-search

(c) Waveform for max-search

Fig. 5: **Illustration of bit-serial min/max search.** (a) Example of min/max-search among six 8-bit unsigned integers by iteratively comparing the bits from MSB to LSB; (b) waveforms of min-search by setting SEL low, where the output RES is Data[3]; (c) waveforms of max-search by setting SEL high, where the output RES is Data[1].

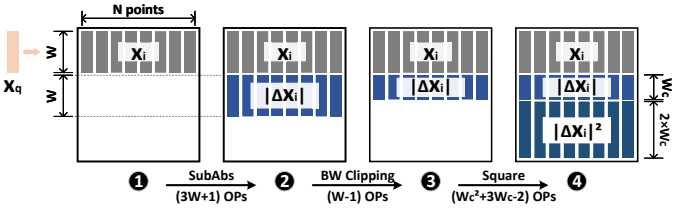


Fig. 6: **Distance calculation along the x-dimension,** encompassing three bit-serial-based functions: subtraction and absolute value (SubAbs), bit-width (BW) clipping, and square. W is the initial bit-width, W_c is the bit-width after clipping.

the latch bits (M) into an index using a priority encoder, enabling identification of the indices of the searched reference points.

C. Execution Flow

The execution flow of PICK follows the dataflow illustrated in Fig. 3, leveraging a two-stage pipeline to parallelize distance calculation and top-k search operations. The detailed implementation of these processes is outlined as follows.

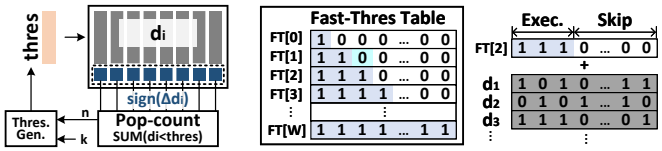
1) Distance Calculation with Bit-width Clipping: The distance calculation determines the Euclidean distances between reference points and a given query point, incorporating bit-width clipping as described in Section III-B. The coordinates $(\boldsymbol{x}_i, \boldsymbol{y}_i, \boldsymbol{z}_i)$ of the reference points are stored as vectors in three BS-PIM arrays, utilizing a vertically aligned bit layout [28], as illustrated in Fig. 6. The query point coordinates are loaded into registers and broadcast to the BS-PIM arrays [31] for parallel computation.

Distance computation along each dimension involves three key bit-serial arithmetic functions, as shown in Fig. 6: subtraction and absolute value (SubAbs), bit-width clipping, and squaring. The implementation details of the bit-width clipping are provided in Algorithm 1. This technique introduces a complexity of $O(N)$, which simplifies the subsequent squaring operation, reducing its computational burden from $O(N^2)$. Finally, the summation of distance vectors across the x, y, and z dimensions is performed using bit-serial addition units, as depicted

Algorithm 1: Bit-serial-based Bit-width Clipping.

Input: A W-bit unsigned integer D; target bit-width W_c.

1: **for** each $i \in [W-1 : W_c+1]$ **do**
2: $\quad D[i] = OR(D[i], D[i-1])$
3: **for** each $i \in [0 : W_c]$ **do**
4: $\quad D[i] = OR(D[i], D[W_c])$
5: $D = D[0 : W_c]$



(a) Filtering-bases Search (b) Skip bit-serial addition with fast-thresholds

Fig. 7: **Illustration of filtering-based top-k search.** (a) The threshold is dynamically adjusted to make the number of points within the threshold approximates k. (b) Special thresholds (continuous '1's from MSB) are used to bypass the bit-serial additions of lower bits.

in Fig. 3. These addition units comprise digital 4-2 compressors and latches to achieve efficient accumulation of partial results.

2) Filtering-and-Selection-based Top-k Search: For small values of k, PICK performs top-k search by iteratively selecting the point with the smallest distance using min-search (Fig. 5b). To prevent previously selected points from being reconsidered in subsequent iterations, an additional bit is appended to the MSB of each reference point's distance. This bit acts as a mask, ensuring that selected points are assigned non-minimal distances and excluded from future searches.

For larger values of k, a filtering approach is employed to efficiently identify a subset of n points, where n approximates k within a predefined margin, such that $|n-k| < m$. The filtering-based top-k search process is depicted in Fig. 7a. During this process, the threshold generator produces a dynamic threshold value. The BS-PIM performs parallel bit-serial subtraction $\Delta d_i = (d_i - thres)$ and retains the sign bits. To simplify operations, negative thresholds are used, converting subtraction to addition in two's complement representation.

The pop-count unit sums the sign bits to determine the #points within the threshold ($d_i < thres$), denoted as n. Based on the comparison between n and k, the threshold generator adjusts the threshold iteratively: increasing $thres$ if $n < k$ or decreasing $thres$ if $n > k$. To accelerate threshold updates, a fast-threshold table is introduced, as shown in Fig. 7b. The table comprises threshold values consisting of continuous '1' bits starting from the MSB, enabling the system to bypass bit-serial additions for lower bits and save computational cycles. Thresholds from this table, denoted as *fast thresholds* (FT), are sequentially applied until $FT[x]$ results in $n < k$. Subsequently, thresholds are refined through binary approximation between $FT[x-1]$ and $FT[x]$, continuing until the condition $|n-k| < m$ is satisfied. Once the filtering phase concludes, the system applies either min-search (if $n < k$) or max-search (if $n > k$) to include or exclude the remaining $|n-k|$ reference points, finalizing the top-k search.

3) Pipeline Design: The pipeline of PICK is meticulously designed to optimize performance and maximize hardware utilization. Example pipeline and dataflow designs for distance calculation and top-k search are illustrated in Fig. 8. The operations are color-coded for clarity: memory read/write operations are marked in gray, while computational tasks are highlighted in green. The latency for distance calculation is constant, determined by the fixed bit-width and clipping width configurations. In contrast, the latency for the top-k search varies depending on the number of points processed, as shown in Fig. 2b.

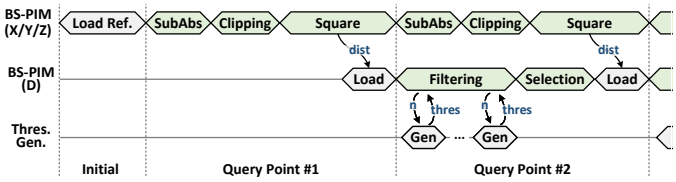


Fig. 8: **Pipeline design** of distance calculation and top-k search to hide latency and improve throughput.

TABLE I: Evaluation Benchmarks.

Dataset	Scenario	Sensor	Resolution	#Refer. Points	#Query Points
KITTI [20]	Outdoor	LiDAR	10^{-2}m	64k	64k
SONN [21]	Object	3D Scanner	10^{-4}m	64k	64k
S3DIS [22]	Indoor	3D Scanner	10^{-3}m	512k	512k
DALES [23]	Urban	LiDAR	10^{-2}m	512k	512k

V. Evaluation

A. Methodology

Benchmarks and Baselines. We construct evaluation benchmarks using four widely recognized real-world point cloud datasets, as summarized in Table I. These datasets encompass diverse application scenarios, including objects, indoor environments, outdoor settings, and urban landscapes, providing a comprehensive range of resolutions. The KITTI [20] and ScanObjectNN (SONN) [21] datasets represent smaller-scale scenarios, containing up to 64k points per frame. In contrast, S3DIS [22] and DALES [23] represent larger-scale environments, with up to 512k points per frame. Across all four datasets, the x, y, and z coordinates are encoded as 16-bit signed integers. For evaluation, we compare PICK against an RTX 3090 GPU (using PyG [35]) and prior kNN accelerators, including QuickNN [11], ParallelNN [12], and BitNN [10], which serve as our baselines.

Implementations and Specifications. We implement PICK in 28nm technology and comprehensively validate its critical functionalities. The SRAM is generated using a commercial memory compiler and integrated with customized computing circuits using Virtuoso. For the data buffers supporting bit-layout transposition, we adopt the performance metrics reported in [30]. The digital circuits are implemented in Verilog and synthesized using Design Compiler. Architectural overheads, including data paths and memory organization, are modeled using NVSim [36], while off-chip memory access is evaluated with DRAMSim3 [37]. Hardware configurations and performance specifications are detailed in Table II, where PICK is evaluated in two configurations: small (S), optimized for smaller datasets (KITTI and SONN), and large (L), tailored for larger datasets (S3DIS and DALES). The hardware specifications are input into an in-house, cycle-level simulator to assess overall performance, where a PyTorch-based arithmetic library is developed to emulate hardware behaviors and evaluate accuracy, latency, and energy.

B. Experiment Results

Accuracy Analysis is conducted across four datasets with varying values of k (ranging from 1 to 64) and clipped bit-width (ranging from 1 to 12), to demonstrate that our proposed bit-width clipping method does not introduce noticeable accuracy degradation under appropriate configurations. The results are presented in Fig. 9, where accuracy below 80% is colored in red, while accuracy exceeding 99.9999998% (6σ, considered effectively error-free) is colored in gray. As evident from the results, maintaining accuracy for larger values of k requires more bits. However, for configurations clipping 5 bits on KITTI and DALES, 6 bits on SONN, and 8 bits on S3DIS, no noticeable accuracy loss is observed, even for 64-NN. These configurations

TABLE II: Hardware Configurations and Performance Specifications

Component	Configurations	Area (mm^2)	Power (mW)
PE	(96+18)kb BS-PIM, 512 parallelism	0.024	3.503
Core	32 PEs, 32kb Buffer, 16k parallelism	0.799	114.8
PICK(S)	4 Cores, 64k parallelism, 1GHz	3.293	463.7
PICK(L)	32 Cores, 512k parallelism, 1GHz	25.78	3677

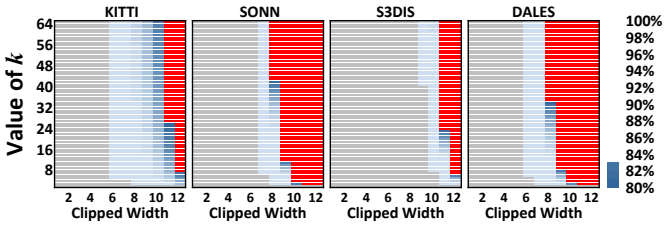
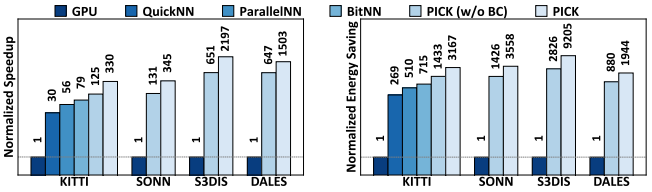
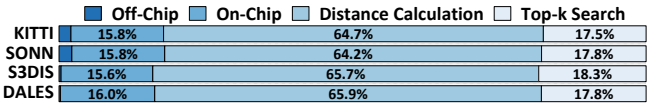


Fig. 9: **Accuracy map of PICK for varying values of k and clipped bit-width (initially 16-bit)**. Accuracy below 80% is filled in red, while accuracy exceeding 99.9999998% (6σ) is filled in gray.



(a) Normalized Speedup and Energy Saving



(b) Energy Breakdown of PICK

Fig. 10: **Normalized comparisons and energy breakdown of 5-NN search**. The bit-width clipping feature is disabled in PICK (w/o BC) to highlight the performance improvements.

achieve a significant reduction in the operational cost of distance calculation, ranging from 38% to 56%, as illustrated in Fig. 1(c).

Performance Analysis. Following prior works, we perform 5-NN searches on the four datasets to evaluate the performance of PICK, and compare it with GPU, QuickNN [11], ParallelNN [12], and BitNN [10]. The clipped bit-widths for the datasets are configured as $\{8,9,11,8\}$-bit, yielding accuracies of 99.38%, 99.94%, 99.73%, and 99.89%, respectively. To emphasize the effectiveness of our proposed bit-width clipping, we disable bit-width clipping in PICK (w/o BC). For fair comparisons, all non-GPU designs are normalized to the same power level. As shown in Fig. 10a, PICK achieves a 4.17× speedup and a 4.42× energy saving compared to the state-of-the-art design, BitNN, on the KITTI dataset. Furthermore, the bit-width clipping technique contributes to an additional 2.74× speedup and 2.54× energy savings. The energy breakdown, depicted in Fig. 10b, reveals that distance calculation and top-k search dominate the energy consumption. However, the costs of on-chip and off-chip data movements are minimized by the efficient PIM architecture and optimized execution flow, contributing to the overall efficiency.

Ablation Study. Fig. 11 presents the latency and energy scaling of PICK across different values of k, with bit-width clipping (BC), filtering-and-selection-based search (FSS), and fast-threshold (FT) individually disabled. For each value of k, the smallest clipping width is selected to maintain an accuracy above 99%. The stepwise jumps in the curves are primarily attributed to changes in the clipping width. Overall, BC demonstrates the most substantial impact on performance

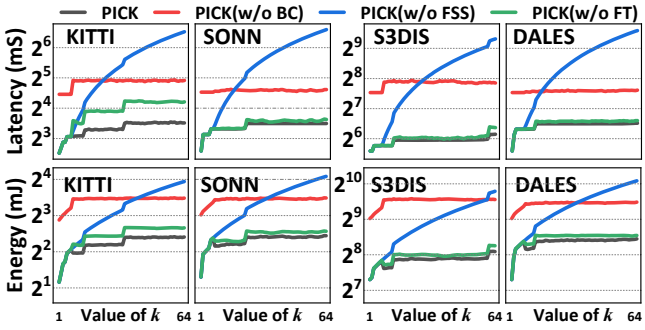


Fig. 11: **Latency and energy scaling** of PICK when disabling bit-width clipping (BC), filtering-and-selection-based search (FSS), and fast-threshold (FT), with accuracy constrained above 99%.

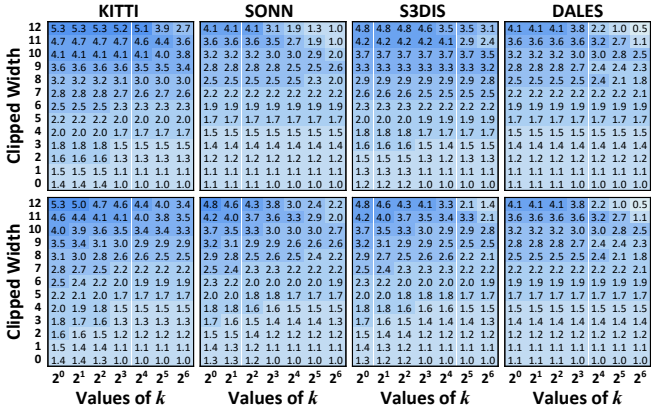


Fig. 12: **Latency (top) and energy (bottom) improvements** of PICK for varying values of k and clipped bit-width (initially 16-bit).

and energy efficiency. However, as k increases, FSS emerges as the dominant factor driving further optimizations. These results highlight the complementary contributions of the proposed techniques in scaling latency and energy efficiency of PICK across different configurations.

Data Movements Reduction. The data movement of PICK is significantly reduced, owing to its efficient architecture and optimized data flow. For example, on the KITTI dataset, BitNN [10] incurs 5.4MB of DRAM access, while PICK requires only 1.4MB for initialization and result storage, achieving over 70% reduction by fully eliminating runtime off-chip access. Additionally, on-chip access to buffers and registers is reduced from 0.4GB to 0.28GB.

Design Space Exploration. Fig. 12 illustrates the latency (top) and energy (bottom) improvements of PICK across varying values of k and clipped bit-width, using the performance of 64-NN without bit-width clipping as the baseline. PICK can achieve up to 5.29× speed and 5.25× energy saving. When combined with the accuracy map in Fig. 9, the optimal configuration can be selected based on specific task requirements. For example, applications demanding high accuracy, such as autonomous driving, may adopt lower clipped bit-widths, whereas latency-sensitive tasks, such as AR/VR applications, can benefit from higher clipped bit-widths for improved performance.

VI. Conclusion

This work proposes PICK, the first PIM architecture for kNN search in point clouds, incorporating novel bit-width clipping technique and filtering-and-selection-based search strategy to optimize the distance calculation and top-k search. Compared to existing designs, PICK achieves remarkable performance improvements.

REFERENCES

[1] Behnam Behroozpour, Phillip AM Sandborn, Ming C Wu, and Bernhard E Boser. Lidar system architectures and circuits. *IEEE Communications Magazine*, 55(10):135–142, 2017.

[2] Ying Li, Lingfei Ma, Zilong Zhong, Fei Liu, Michael A Chapman, Dongpu Cao, and Jonathan Li. Deep learning for lidar point clouds in autonomous driving: A review. *IEEE Transactions on Neural Networks and Learning Systems*, 32(8):3412–3432, 2020.

[3] Gang Wang, Wenlong Li, Cheng Jiang, Dahu Zhu, Zhongwei Li, Wei Xu, Huan Zhao, and Han Ding. Trajectory planning and optimization for robotic machining based on measured point cloud. *IEEE transactions on robotics*, 38(3):1621–1637, 2021.

[4] Daniel Garrido, Rui Rodrigues, A Augusto Sousa, Joao Jacob, and Daniel Castro Silva. Point cloud interaction and manipulation in virtual reality. In *2021 5th International Conference on Artificial Intelligence and Virtual Reality (AIVR)*, pages 15–20, 2021.

[5] Erik Sandström, Yue Li, Luc Van Gool, and Martin R Oswald. Point-slam: Dense neural point cloud-based slam. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18433–18444, 2023.

[6] Zhengyou Zhang. Iterative closest point (icp). In *Computer vision: a reference guide*, pages 718–720. Springer, 2021.

[7] Hengshuang Zhao, Li Jiang, Chi-Wing Fu, and Jiaya Jia. Pointweb: Enhancing local neighborhood features for point cloud processing. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5565–5573, 2019.

[8] Renrui Zhang et al. Nearest neighbors meet deep neural networks for point cloud analysis. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1246–1255, 2023.

[9] Ziyu Ying, Sandeepa Bhuyan, Yan Kang, Yingtian Zhang, Mahmut T Kandemir, and Chita R Das. Edgepc: Efficient deep learning analytics for point clouds on edge devices. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–14, 2023.

[10] Meng Han, Liang Wang, Limin Xiao, Hao Zhang, Tianhao Cai, Jiale Xu, Yibo Wu, Chenhao Zhang, and Xiangrong Xu. Bitnn: A bit-serial accelerator for k-nearest neighbor search in point clouds. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1278–1292. IEEE, 2024.

[11] Reid Pinkham, Shuqing Zeng, and Zhengya Zhang. Quicknn: Memory and performance optimization of kd tree based nearest neighbor search for 3d point clouds. In *2020 IEEE International symposium on high performance computer architecture (HPCA)*, pages 180–192. IEEE, 2020.

[12] Faquan Chen, Rendong Ying, Jianwei Xue, et al. Parallelnn: A parallel octree-based nearest neighbor search accelerator for 3d point clouds. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 403–414. IEEE, 2023.

[13] Zihan Liu, Wentao Ni, Jingwen Leng, Yu Feng, Cong Guo, Quan Chen, Chao Li, Minyi Guo, and Yuhao Zhu. Juno: Optimizing high-dimensional approximate nearest neighbour search with sparsity-aware algorithm and ray-tracing core mapping. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 549–565, 2024.

[14] Yangfan Li, Mengquan Li, Cen Chen, Xiaofeng Zou, Hongen Shao, Fengxiao Tang, and Kenli Li. Simdiff: Point cloud acceleration by utilizing spatial similarity and differential execution. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.

[15] Chengliang Wang, Zhetong Huang, Ao Ren, et al. An fpga-based knn seach accelerator for point cloud registration. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE, 2024.

[16] Yunhao Hu, Hao Sun, Chunxu Guo, Qi Deng, and Yajun Ha. An energy-efficient and fast knn search accelerator for large scale point cloud map. In *2023 30th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, pages 1–4. IEEE, 2023.

[17] Yejin Lee, Hyunji Choi, Sunhong Min, Hyunseung Lee, Sangwon Beak, Dawoon Jeong, Jae W Lee, and Tae Jun Ham. Anna: Specialized architecture for approximate nearest neighbor search. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 169–183. IEEE, 2022.

[18] Batcher. Bit-serial parallel processing systems. *IEEE Transactions on Computers*, 100(5):377–384, 1982.

[19] RoboSense. 128-beam lidar customized for l4 autonomous vehicle. https://www.robosense.cn/en/rslidar/RS-Ruby_Plus.

[20] Andreas Geiger et al. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3354–3361. IEEE, 2012.

[21] Mikaela Angelina Uy, Quang-Hieu Pham, Binh-Son Hua, Thanh Nguyen, and Sai-Kit Yeung. Revisiting point cloud classification: A new benchmark dataset and classification model on real-world data. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1588–1597, 2019.

[22] Iro Armeni, Ozan Sener, Amir R Zamir, Helen Jiang, Ioannis Brilakis, Martin Fischer, and Silvio Savarese. 3d semantic parsing of large-scale indoor spaces. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1534–1543, 2016.

[23] Nina Varney, Vijayan K Asari, and Quinn Graehling. Dales: A large-scale aerial lidar data set for semantic segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pages 186–187, 2020.

[24] Naveen Verma, Hongyang Jia, et al. In-memory computing: Advances and prospects. *IEEE Solid-State Circuits Magazine*, 11(3):43–55, 2019.

[25] Donghyuk Kim, Chengshuo Yu, Shanshan Xie, Yuzong Chen, Joo-Young Kim, Bongjin Kim, Jaydeep P Kulkarni, and Tony Tae-Hyoung Kim. An overview of processing-in-memory circuits for artificial intelligence and machine learning. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 12(2):338–353, 2022.

[26] Hidehiro Fujiwara, Haruki Mori, Wei-Chang Zhao, Kinshuk Khare, Cheng-En Lee, Xiaochen Peng, Vineet Joshi, Chao-Kai Chuang, Shu-Huan Hsu, Takeshi Hashizume, et al. 34.4 a 3nm, 32.5 tops/w, 55.0 tops/mm 2 and 3.78 mb/mm 2 fully-digital compute-in-memory macro supporting int12× int12 with a parallel-mac architecture and foundry 6t-sram bit cell. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 67, pages 572–574. IEEE, 2024.

[27] Yiyang Yuan, Yiming Yang, Xinghua Wang, Xiaoran Li, Cailian Ma, Qirui Chen, Meini Tang, Xi Wei, Zhixian Hou, Jialiang Zhu, et al. 34.6 a 28nm 72.12 tflops/w hybrid-domain outer-product based floating-point sram computing-in-memory macro with logarithm bit-width residual adc. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 67, pages 576–578. IEEE, 2024.

[28] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramaniyan, Ravi Iyer, Dennis Sylvester, David Blaaauw, and Reetuparna Das. Neural cache: Bit-serial in-cache acceleration of deep neural networks. In *2018 ACM/IEEE 45Th annual international symposium on computer architecture (ISCA)*, pages 383–396. IEEE, 2018.

[29] Daichi Fujiki, Scott Mahlke, and Reetuparna Das. Duality cache for data parallel acceleration. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 397–410, 2019.

[30] Jingcheng Wang, Xiaowei Wang, et al. A 28-nm compute sram with bit-serial logic/arithmetic operations for programmable in-memory vector computing. *IEEE Journal of Solid-State Circuits*, 55(1):76–86, 2019.

[31] Chen Nie, Chenyu Tang, Jie Lin, Huan Hu, Chenyang Lv, Ting Cao, Weifeng Zhang, Li Jiang, Xiaoyao Liang, Weikang Qian, et al. Vspim: Sram processing-in-memory dnn acceleration via vector-scalar operations. *IEEE Transactions on Computers*, 2023.

[32] Nastaran Hajinazar, Geraldo F Oliveira, Sven Gregorio, João Dinis Ferreira, Nika Mansouri Ghiasi, Minesh Patel, Mohammed Alser, Saugata Ghose, Juan Gómez-Luna, and Onur Mutlu. Simdram: A framework for bit-serial simd processing using dram. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 329–345, 2021.

[33] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *Advances in neural information processing systems*, 30, 2017.

[34] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics (tog)*, 38(5):1–12, 2019.

[35] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[36] Xiangyu Dong, Cong Xu, Yuan Xie, and Norman P Jouppi. Nvsim: A circuit-level performance, energy, and area model for emerging nonvolatile memory. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 31(7):994–1007, 2012.

[37] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. Dramsim3: A cycle-accurate, thermal-capable dram simulator. *IEEE Computer Architecture Letters*, 19(2):106–109, 2020.